

Developer Evaluation Task

Organization-Scoped Skill Registry Vertical Slice

JARVIS AI COO

8-HOUR TECHNICAL EVALUATION

Purpose

A privacy-first, local-first, multi-tenant backend evaluation designed to assess architecture, authorization, versioning, debugging, testing and engineering ownership without exposing private production code, credentials or company data.

Title	Organization-Scoped Skill Registry Vertical Slice
Deadline	8 hours from the developer's start time
Submission	Developer's own GitHub repository
Frontend	Not required
External AI/API	Not required
Primary focus	Tenant isolation, authorization, immutable skill versioning and auditability

Evaluation principle: The task should be completed independently. Architecture, coding, debugging, test design and documentation are part of the assessment.

1. Goal

Build a working backend prototype where multiple organizations can create, review and activate custom AI COO skills while preserving strict organization-level isolation. Organization A must not be able to access, modify or activate Organization B's data.

2. Fixture Organizations

- **ABC Construction**
- **XYZ Builders**

3. Required End-to-End Workflow

```
Authenticated organization
→ manual skill draft create
→ draft review
→ owner activation
→ active skill retrieve
→ exact version audit record
```

4. Core Requirements

- FastAPI backend.
- PostgreSQL is preferred. SQLite is allowed only with a written justification.
- Canonical ownership key: **organization_id**.
- Skill lifecycle: **draft** → **active** → **disabled**.
- An active skill must never be modified in place; changes create a new immutable version.
- Only an organization owner can activate a skill.
- Organization A cannot read, update or activate Organization B's skill.
- A skill's requested tools must not grant permissions automatically.
- Skill version creation and activation must be audit logged.
- Inputs must be validated and failures must return explicit, useful errors.
- No secrets, credentials or real company/customer data may be committed.

5. Minimum API Capabilities

Exact route names are the developer's choice. The system must support:

- Create a skill draft.
- List skills belonging to the current organization.
- Read one skill together with its versions.
- Create a new immutable version.
- Activate an approved version.
- Disable a skill.
- Retrieve active skills for a department.

6. Mandatory Automated Tests

- Same-organization create/read succeeds.
- Cross-organization read is denied.
- Cross-organization update is denied.
- Non-owner activation is denied.
- Draft skill cannot execute or load as active.
- Disabled skill is excluded from runtime selection.
- Active version is immutable.
- Duplicate activation request is safe and idempotent.
- Invalid or destructive requested tool is rejected.
- Audit record contains organization, actor, event and version.

7. Submission Requirements

The developer's GitHub repository must include:

- Source code.
- Database schema and/or migrations.
- Automated tests.
- Docker Compose startup.
- .env.example containing placeholders only.
- README with setup instructions and API examples.
- A short architecture decision note.
- Test output.
- Known limitations.
- Meaningful commit history.

8. Final Report Format

Repository URL:
Start time:
Finish time:
Approximate hours:
Final commit SHA:

Goal achieved:
Architecture decisions:
Tests passed:
Security/isolation evidence:
Known limitations:
What I would implement next:
AI tools used, if any:

9. Restrictions

- No copied proprietary Gobind Trades code.
- No real customer or company data.
- No hardcoded secrets.
- No frontend required.
- No external AI/model API required.
- No automatic skill activation.
- No cross-tenant admin shortcut.
- Request permission before expanding the evaluation scope.

10. Review Score — 100 Marks

Category	Marks
Tenant isolation and authorization	30
Correct domain/version lifecycle	20
Tests and failure handling	20
Code architecture/readability	15
Setup and documentation	10
Git discipline and final report	5
TOTAL	100

11. Automatic Rejection Criteria

Automatic rejection: committed secret, cross-tenant leakage, fake tests, application fails to start, or an active skill can be silently mutated.

12. What This Evaluation Measures

This task is intentionally small in feature count but deep in engineering judgment. A strong candidate should demonstrate secure tenant ownership boundaries, thoughtful domain modeling, immutable version management, reliable tests, clear failure behavior, reproducible local startup, and disciplined documentation.

The objective is not to build the complete Jarvis AI COO. The objective is to prove that the developer can safely implement one production-relevant vertical slice of a multi-tenant AI COO platform.